AUDIT ZIP ZONEFLOW (12)

Lampiran teknis untuk implementasi AZBO versi terbaru

Sumber: arsip pengguna zoneflow (12).zip • Pemeriksaan kode dan pengujian lokal • 28 Oktober 2026

Keputusan singkat

Arsip ini adalah basis website ZoneFlow yang dapat dikembangkan, tetapi belum mengintegrasikan mesin AZBO terbaru yang sudah disepakati tim. Pengujian yang lulus terutama memverifikasi implementasi lama, bukan pemenuhan tiga persyaratan baru.

Komponen utama	Status ZIP	Prioritas
Antrean satu lorong	Belum sesuai: ketersediaan sel/segmen terlalu dini	P0
GA dengan jumlah zona berubah	Belum sesuai: K=3, zona 5–15 baris	P0
Rekomendasi Monte Carlo per input	Belum sesuai: alur utama 1 replikasi, frontend lokal	P0
Keterisian penuh & bagasi input	Parsial: UI membolehkan n < kapasitas	P1
Berbagai pesawat	Parsial: frontend beragam; backend 30×6	P1
Deploy backend	Belum terbukti; GitHub Pages hanya frontend	P1

Hasil uji yang sudah dijalankan

- Backend Python: 37 tes lulus dan 116 subtes lulus dalam ±9 detik. Tiga tes GA berat sengaja dikecualikan pada pengujian terarah karena suite penuh sebelumnya melewati batas waktu 120 detik; hasil tiga tes tersebut belum diklaim lulus.
- Kode referensi AZBO baru (paket terpisah): 11 dari 11 pengujian unit lulus. Ini belum berarti kode referensi sudah terhubung dengan frontend ZoneFlow.
- Reproduksi langsung bug antrean: dua penumpang ke baris 3 tercatat menggunakan segmen lorong baris 1 secara bersamaan pada interval t=1,0 hingga 2,4 detik (overlap 1,4 detik), jika segmen dianggap kapasitas satu.
- Pembangunan/pengujian frontend belum selesai: pemasangan dependensi npm gagal di lingkungan pengujian; karena itu lint, build, dan tes browser frontend belum terverifikasi pada arsip ini.

Bukti kode yang paling menentukan

Lokasi	Bukti temuan
backend/app/config.py	K_ZONES=3; MIN_ROWS_PER_ZONE=5; MAX_ROWS_PER_ZONE=15; WALK_PER_ROW_S=2.4; ENTRY_GAP_S=1.0.
backend/app/simulation_core.py:345–352	Segmen lorong dinyatakan tersedia pada start + aisle_headway_s, padahal penumpang keluar pada start + walk_s.
backend/app/simulation_core.py:1138+	GA nyata sudah tersedia, tetapi kromosom dan pencarian masih untuk tiga zona.
src/App.tsx:176–184	Tombol utama memanggil runAzboOptimizationForScenario() lokal, bukan API GA Python.
src/simulation/engine.ts:1243–1267	Data penumpang dibuat sekali; 3 zona di-enumerasi dan

	dievaluasi satu kali tiap kandidat.
src/simulation/engine.ts:1300+	Rekomendasi diranking menurut satu boarding_time_seconds; GA-Zone-WMA memakai konfigurasi lama pada baseline.
backend/app/schemas.py:1–45	Input backend masih menerima load_factor < 1 dan zona custom yang harus tepat dua batas.
.github/workflows/deploy.yml	Hanya build frontend; tidak menjalankan layanan Python FastAPI.

Spesifikasi AZBO yang harus tetap

- AZBO = Aircraft Zone-Based Boarding Optimization; kerangka optimasi yang memilih strategi dan konfigurasinya untuk satu skenario, bukan satu strategi tetap.
- Skenario penelitian: satu lorong/pintu depan pada prototipe, keterisian kursi 100%; proporsi pembawa bagasi merupakan input deterministik; identitasnya diacak dengan jumlah tetap pada setiap replikasi.
- Jumlah zona K dibangkitkan sebagai kandidat acak dan dioptimalkan; setiap kandidat memiliki K tetap ketika disimulasikan. Batas struktural $1 \le K \le N$ berasal dari jumlah baris, bukan batas buatan.
- Objektif: meminimalkan E[T_boarding | kondisi, strategi]. GA mencari solusi terbaik yang ditemukan; bukan jaminan optimum global.
- Simulator utama Python; frontend React berfungsi sebagai antarmuka, tampilan progres, dan animasi berdasarkan keluaran backend.

Instruksi perbaikan: tiga masalah P0

P0-A — Mesin antrean

Ganti logika aisle_free_at per baris dengan event/sel yang secara eksplisit mencatat penempatan, perpindahan, dan pelepasan ruang. Rujukan implementasi awal: AZBO_Prototype_Reference/azbo_core.py. Tetapkan panjang sel, panjang baris, dan daya tampung fisik secara eksplisit. Validasi invariant: tidak ada dua penumpang pada sel berkapasitas satu pada interval waktu yang sama; tidak saling mendahului; jika depan tertahan, belakang tetap menempati sel sebelumnya. Jangan gunakan jeda masuk 1 detik sebagai pemaksaan dasar.

P0-B — GA variabel K

Gunakan N−1 gen biner batas antarzona (cut vector). Jumlah zona K=1+sum(cuts). Setiap zona berisi sedikitnya satu baris; bangkitkan K secara probabilistik untuk inisialisasi dan mutasi, lalu kunci K saat evaluasi kandidat. Bentuk urutan zona sebagai permutasi valid, regenerasi/repair setelah crossover dan mutasi. Evaluasi setiap keluarga strategi sesuai parameter yang memang berlaku; pertahankan WMA, Back-to-Front, dan Random sebagai pembanding.

P0-C — Rekomendasi simulasi berulang

Untuk skenario aktif yang dimasukkan pengguna, gunakan replikasi pelatihan dengan common random numbers saat menghitung fitness; setelah GA selesai, uji kandidat final dan baseline pada replikasi baru. Kembalikan rata-rata, simpangan baku, selang kepercayaan, perbedaan berpasangan, seed, konfigurasi zona, dan batasan simulator. Ranking rekomendasi didasarkan pada rata-rata pengujian, bukan satu waktu simulasi. Backend Python menjadi satu-satunya sumber rekomendasi.

Pekerjaan penyambung frontend–backend

- Masukkan mesin referensi ke backend/app secara modular; expose POST /api/v2/azbo/optimize atau endpoint job async dan polling yang setara. Sesuaikan CORS sesuai origin frontend yang dipublikasikan.
- Ubah handleRunSimulationAndAzbo() pada src/App.tsx menjadi pemanggil API, bukan runAzboOptimizationForScenario() lokal. Simpan status running, error, progress, dan hasil. Hindari time-out request panjang; gunakan background job untuk GA besar.

- Jika animasi ingin dipertahankan, backend harus mengirim event trace per penumpang. React hanya menampilkan trace, tanpa mengestimasi ulang posisi menggunakan simulator TypeScript lama.
- Ubah input penumpang menjadi otomatis sama dengan kapasitas kabin; tampilkan pengaturan persentase bagasi dan jumlah pembawa bagasi hasil pembulatan.
- Jangan menggunakan katalog berbagai layout sebagai klaim dukungan fisika multipesawat sebelum backend benar-benar menerima denah dan posisi pintu sesuai layout. Validasi layout satu per satu.
- Catatan deploy: docker-compose.yml menunjuk backend/Dockerfile, sedangkan arsip ini menyediakan backend/Dockerfile.export.txt; perbaiki nama/path sebelum menggunakan Docker Compose.

Kriteria penerimaan dan data yang dibutuhkan

- Unit test: larang dua penumpang dalam sel yang sama, pastikan kursi terisi penuh, jumlah pembawa bagasi deterministik, dan identitasnya acak antar-replikasi.
- Uji algoritma: untuk N kecil, cocokkan hasil GA dengan enumerasi menyeluruh; untuk N besar, periksa validitas kromosom, kemampuan berubahnya K, dan stabilitas terhadap beberapa seed.
- Uji statistik: minimal audit 100 replikasi pengujian per skenario (disesuaikan ketelitian hasil), CRN lintas strategi, split training/testing, dan laporan ketidakpastian.
- Uji website end-to-end: ubah persentase bagasi, jalankan optimasi, pastikan request diterima Python, nilai rekomendasi berubah jika data mendukung, serta hasil animasi cocok dengan trace.
- Data perlu untuk validasi empiris: denah kabin (kursi/baris/pintu), jarak pitch dan kapasitas lorong, titik start/finish boarding, aturan pengumuman grup dan apakah waktunya dihitung, observasi durasi boarding yang sebanding jika tersedia.
- Asumsi awal yang BELUM tervalidasi: model kecepatan Normal terpotong (0,8; 0,1), panjang sel 0,4m, pitch 0,8m, gerakan kursi Triangular (1,8; 2,4; 3), penalti pemanggilan zona nol kecuali ada data. Weibull bagasi 1,7/16 berdasar literatur tetapi perlu cek tumpang tindih definisi durasi.

Status akhir: arsip ZIP ini belum merupakan prototipe final AZBO. Tes lama yang lulus berarti perilaku kode versi lama konsisten dengan harapannya; tes itu belum membuktikan kebenaran fisik model atau pemenuhan spesifikasi baru.